





# Lecture 1 : Introduction à l'IDM et aux enjeux du génie logiciel moderne

## Ingénierie Dirigée par les Modèles (IDM / MDE) – version 2027

**Pr. Aissam BERRAHOU**

Département Ingénierie des systèmes embarqués et mobiles – Smart Systems Lab
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

# Feuille de route



- **Pourquoi modéliser ?** : *Pourquoi ne pas simplement « coder directement » ?*
- **Modèle et IDM** : *Qu'est-ce qu'un modèle, et que change l'IDM ?*
- **Les 4 niveaux de l'OMG** : *Qui décrit qui ? Du feu réel jusqu'au MOF*
- **Exercices** : *Savez-vous classer, construire et vérifier ?*

# Objectifs de la séance

**À la fin de cette séance, vous serez capables de :**

**Expliquer**

pourquoi « coder directement » ne suffit plus pour les systèmes complexes.

**Définir**

un modèle et l'Ingénierie Dirigée par les Modèles (IDM).

**Distinguer**

les 4 niveaux de l'OMG : M0 réel, M1 modèle, M2 méta-modèle, M3 MOF / Ecore.

**Vérifier**

qu'un modèle est conforme à son méta-modèle.

! Un **feu de signalisation** : nous le suivrons du carrefour réel (M0) jusqu'au MOF (M3).

# Pré-requis, outils et lecture des schémas

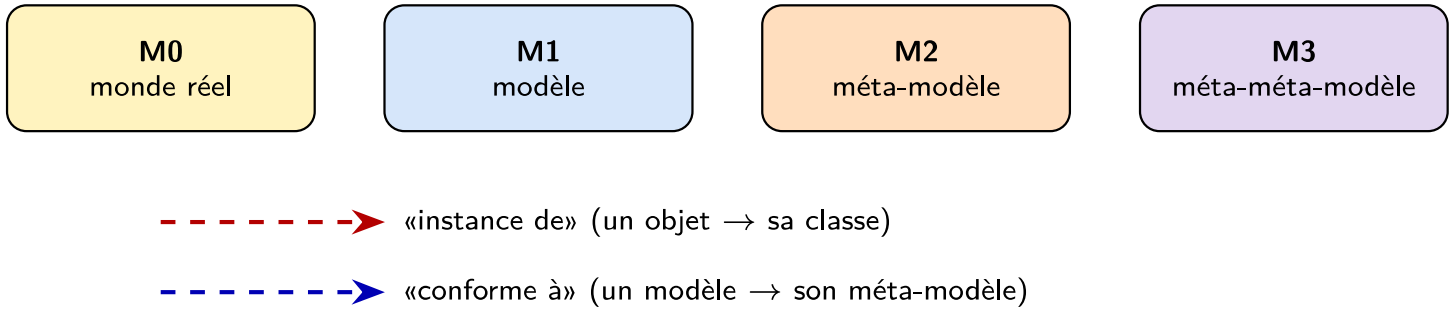
## Pré-requis

- **COO** : UML 2 (classes, objets, cas d'utilisation, états-transitions).
- **POO** : programmation orientée objet en Java.

## Outils

- JDK 17 ou plus ; Eclipse Modeling Tools : **EMF**, **Ecore Tools**.
- **Papyrus** (UML), **Obeo Designer / Sirius**.

## Le même code couleur dans tout le cours



# Pourquoi modéliser ?

*Pourquoi ne pas simplement « coder directement » ?*

# Pourquoi modéliser ?

→ Que représente cette image ?





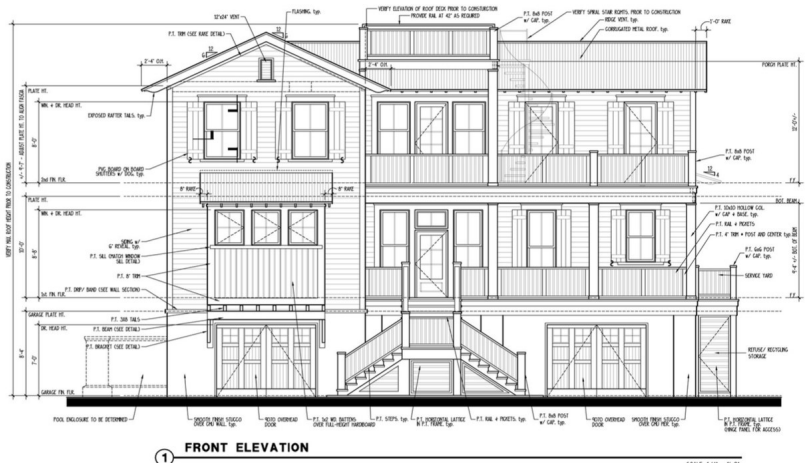
À gauche on croit voir un vrai garage. . . À droite on découvre une **maquette** posée sur une table. Elle **ressemble** au réel, mais on ne peut pas y garer une vraie voiture : c'est un **modèle**.

# Pourquoi modéliser ?

→ Les autres ingénieries modélisent déjà

**Un architecte ne pose pas directement des briques.**

- Il dessine d'abord **plusieurs plans** : masse, coupe, façade, électricité...
- Chaque plan est une **vue** pour un intervenant (client, maçon, électricien).
- Les erreurs sont **détectées sur le plan**, quand elles coûtent encore peu.




## Et le logiciel ?

Pourquoi, si souvent, « on code directement » ?

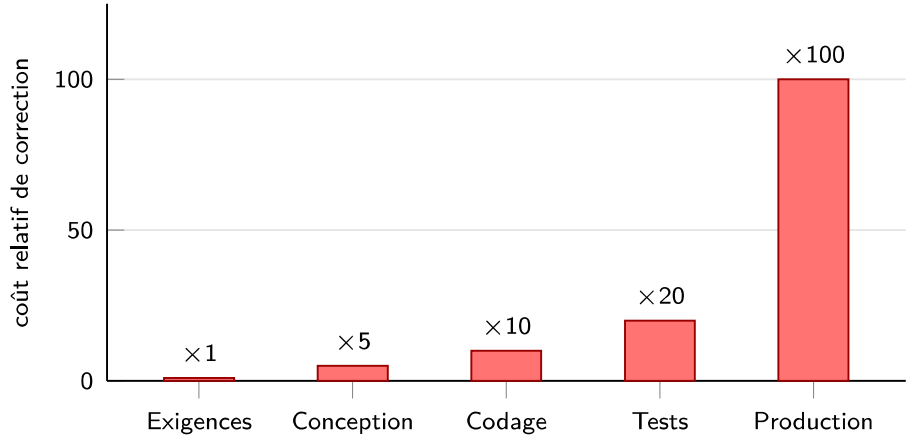
# Pourquoi modéliser ?

→ Coder directement : le « code spaghetti » et son coût

- Pas de **vision globale**, préoccupation **technique** avant le métier.
- Classes **fortement couplées**, responsabilités mélangées.
- Code difficile à **comprendre**, **maintenir**, **réutiliser**.
- La connaissance reste dans la tête du développeur.



## Coût de correction d'une erreur selon la phase




*Ordres de grandeur souvent cités : plus une erreur est découverte tard, plus elle coûte cher.*

# Pourquoi modéliser ?

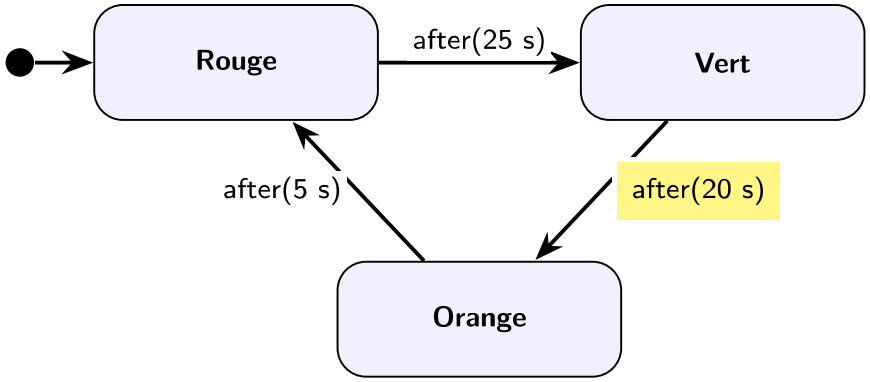
→ Exemple concret : une règle métier noyée dans le code

## Version « code d'abord »

```
class Controleur {
  void cycle() {
    allumer(ROUGE); attendre(25);
    allumer(VERT);  attendre(20);
    allumer(ORANGE); attendre(5);
  }
}
class Simulateur { int dureeVert = 20; ... }
class Rapport { String txt = "vert : 20 s";
    }
```

Le client demande « vert = 30 s » : il faut **retrouver** les 3 endroits... sans en oublier.

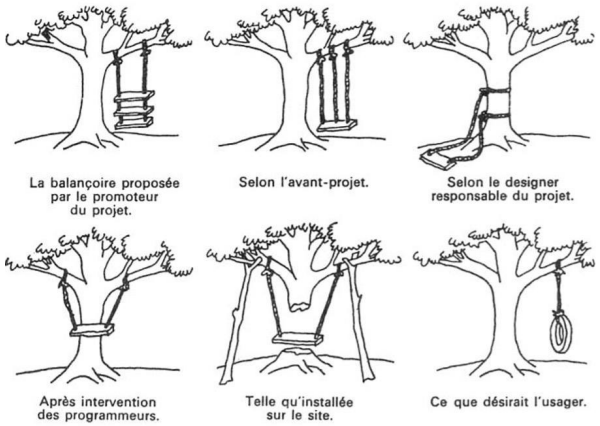
## Version « modèle d'abord »




La durée du vert est écrite **une seule fois**, à un endroit lisible par tous. On la change ici, puis on **régénère** le code.

# Pourquoi modéliser ?

→ La balançoire : chacun comprend le besoin à sa façon




- Client, chef de projet, analyste, programmeur : chacun a **sa** vision du besoin.
- Sans **modèle partagé**, les écarts n'apparaissent qu'à la livraison.

! Le code ne suffit plus pour gérer la complexité ni garantir la **pérennité** (maintenance, réutilisation, évolution). Il faut un niveau d'abstraction supérieur : **le modèle**.

# Pourquoi modéliser ?

→ Question flash

**Question flash**

?

Une erreur de conception découverte en production coûte environ combien de fois plus qu'une erreur corrigée pendant l'analyse des exigences ?

# Pourquoi modéliser ?

→ Question flash

**Question flash**

? Une erreur de conception découverte en production coûte environ combien de fois plus qu'une erreur corrigée pendant l'analyse des exigences ?

**Réponse**

De l'ordre de **100 fois** plus. D'où l'intérêt de **vérifier des modèles** dès le début du projet, quand les corrections sont encore bon marché.

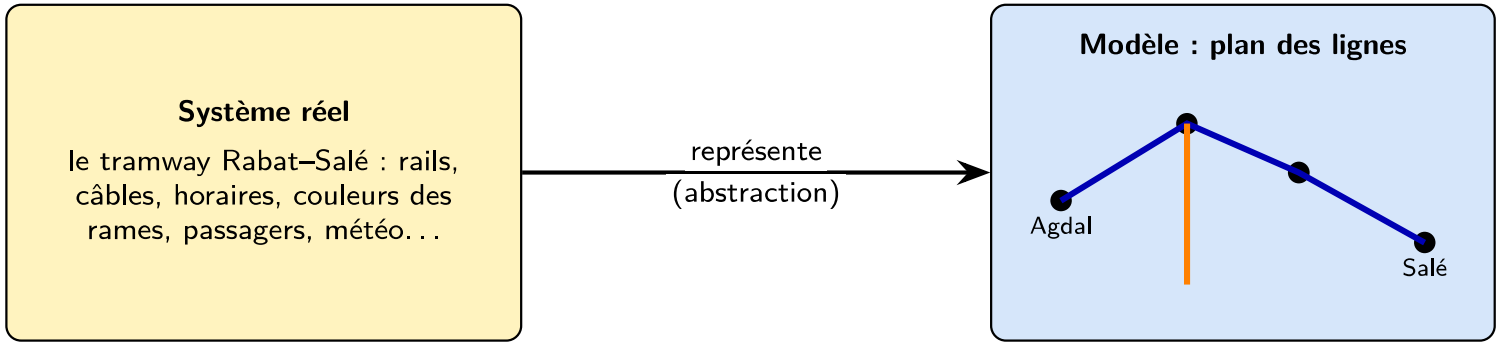
# Modèle et IDM

*Qu'est-ce qu'un modèle, et que change l'IDM ?*

# Modèle et IDM

→ C'est quoi un modèle ?

Selon l'OMG : **« une simplification de la réalité, créée pour mieux comprendre, communiquer, analyser ou simuler un système. »**




**1. Représentation**
il correspond à un système (le tram)

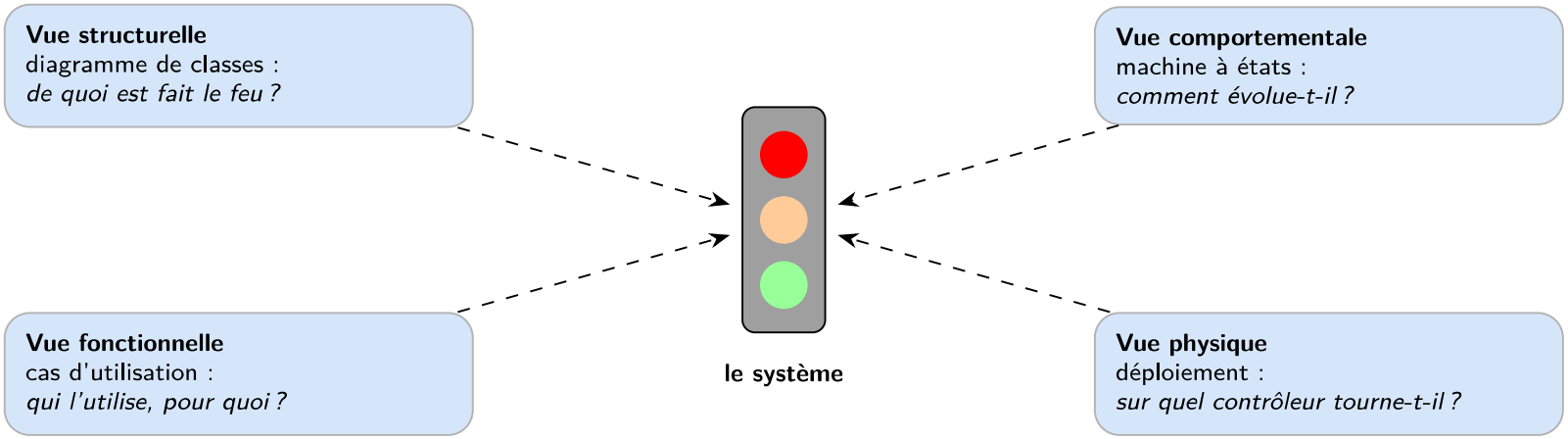
**2. Simplification**
il garde les stations, ignore les câbles

**3. Objectif**
il sert à aller de A à B

**Il est utile ou inutile** pour un objectif. Test : peut-on répondre à une question sur le système en regardant seulement le modèle ?

# Modèle et IDM

→ Un même système, plusieurs modèles




## Selon le rôle

- **Descriptif** : décrit un système existant.
- **Prescriptif** : décrit un système **à construire**.

## Selon l'usage

- **Contemplatif** : un dessin pour comprendre.
- **Productif** : un outil l'exploite (vérification, génération, simulation).

! Passer de modèles **contemplatifs** à des modèles **productifs**.

### Définition

- Approche où **les modèles sont au centre** du développement.
- Ingénierie **générative** : tout ou partie de l'application est **produite automatiquement** à partir des modèles.
- Elle fournit des **langages**, des **concepts** et des **outils** pour créer, vérifier et transformer les modèles.

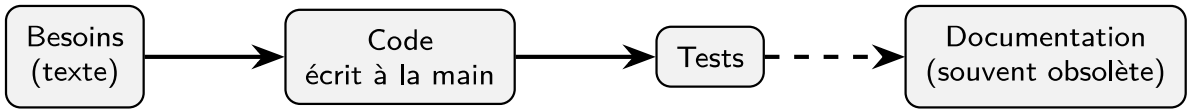
### Apports

- Erreurs détectées **tôt** (~70 % des défaillances naissent des exigences et de la conception).
- Langage **commun** entre experts métier et développeurs.
- **Productivité** par la génération et la réutilisation ; **traçabilité**.

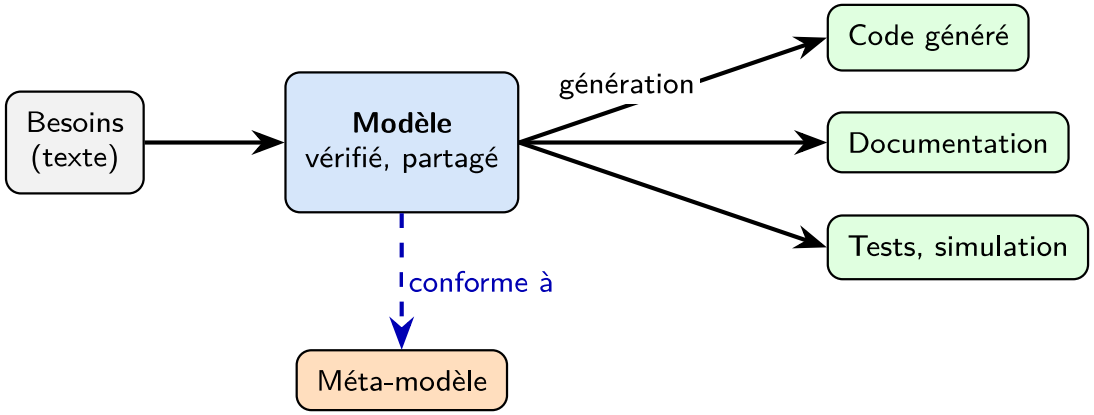
### Limites

Coût d'apprentissage des outils ; un mauvais modèle génère un mauvais code.

**Approche classique : le code est l'artefact central**



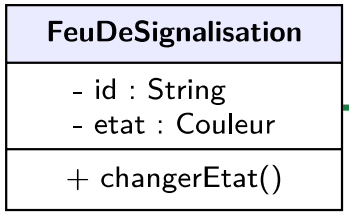

**Approche IDM : le modèle est l'artefact central**



# Modèle et IDM

→ Exemple : du modèle au code

**Modèle (M1)**




génération automatique

**Code Java produit par un générateur**

```java
public class FeuDeSignalisation {
  private String id;
  private Couleur etat;
  public String getId() { return id; }
  public Couleur getEtat() { return etat; }
  public void setEtat(Couleur e) { etat = e; }
  public void changerEtat() {
    // TODO : corps à compléter
  }
}
```

- Le **générateur** écrit le code répétitif (attributs, accesseurs) **sans erreur de frappe**.
- Le développeur se concentre sur le **comportement** (le corps de `changerEtat()`).
- Le modèle change ? On **régénère** : modèle et code restent **synchronisés**.

**Question flash**

?

Un diagramme de classes dessiné au tableau puis effacé à la fin de la réunion est-il un modèle **productif** ?

**Question flash**

?

Un diagramme de classes dessiné au tableau puis effacé à la fin de la réunion est-il un modèle **productif** ?

**Réponse**

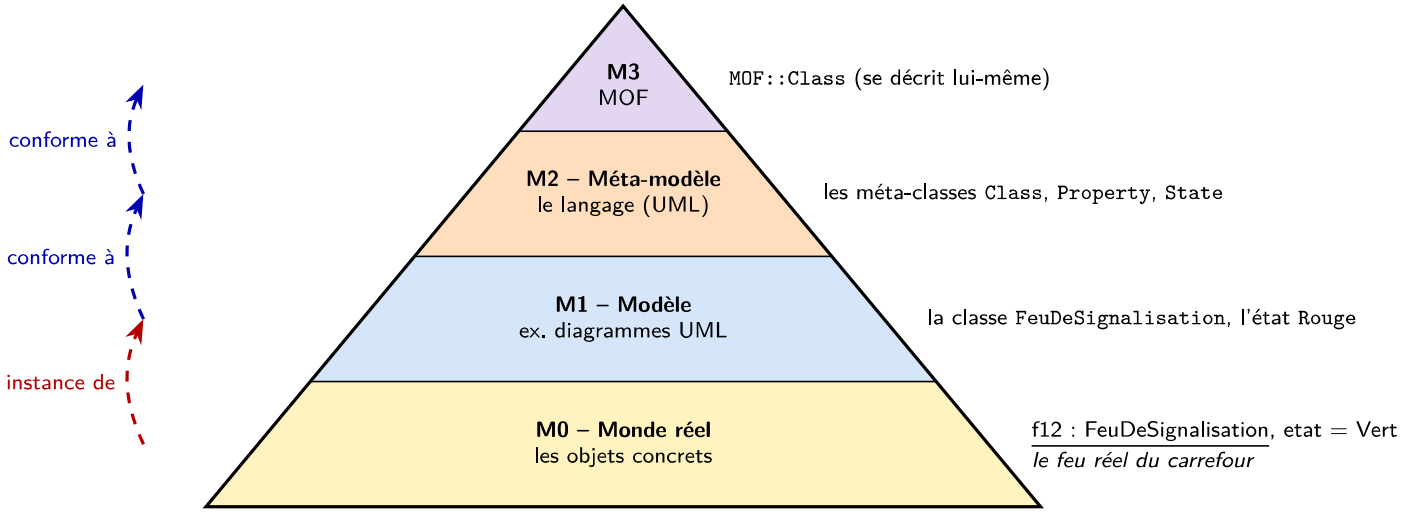
Non : il est **contemplatif** (il a servi à communiquer). Il devient productif lorsqu'un **outil** peut le lire pour le vérifier, le transformer ou générer du code.

# Les 4 niveaux de l'OMG

*Qui décrit qui ? Du feu réel jusqu'au MOF*

# Les 4 niveaux de l'OMG

→ La pyramide de l'OMG




Chaque niveau est décrit par le niveau **au-dessus**. Le MOF (M3) se décrit **lui-même** : la pile s'arrête à 4 niveaux.

# Les 4 niveaux de l'OMG

→ Le même principe dans plusieurs « espaces technologiques »

| Niveau | Monde UML | Monde Java | Monde XML | Monde base de données |
|---|---|---|---|---|
| **M3** | MOF | EBNF (notation des grammaires) | méta-schéma XSD | MOF |
| **M2** | méta-modèle UML | grammaire de Java | schéma XML (.xsd) | méta-modèle relationnel (Table, Column) |
| **M1** | diagramme de classes du feu | le programme Feu.java | document XML (.xml) | schéma de la table FEU |
| **M0** | le feu n° 12 dans la rue | les objets en mémoire à l'exécution | les données du feu n° 12 | les lignes de la table FEU |

! La pile M0 → M3 n'est **pas propre à UML** : c'est une façon générale d'organiser langages et modèles.

# Niveau M0 : le monde réel

→ Les instances, les objets concrets

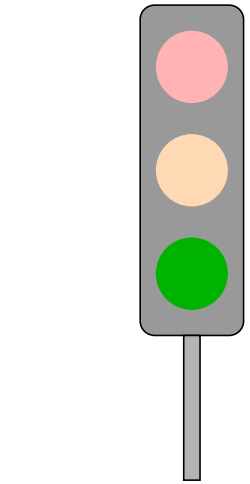


Le niveau M0 contient les **objets réels** (instances) qui existent pendant le fonctionnement du système.

**Exemple**

Le feu **n° 12** du carrefour Hassan II, actuellement **vert**.

> **Daté**, avec des **valeurs réelles** et une **identité** → M0.



décrit par

f12 : FeuDeSignalisation
id = « RBT-012 »
etat = VERT
dureeVert = 20

le feu réel n° 12, carrefour
Av. Hassan II / Av. Allal Ben Abdellah

**M0** = une chose **concrète**, avec des **valeurs réelles**, à un **instant donné**.
Dans 20 s : etat = ORANGE.

# Niveau M1 : le modèle

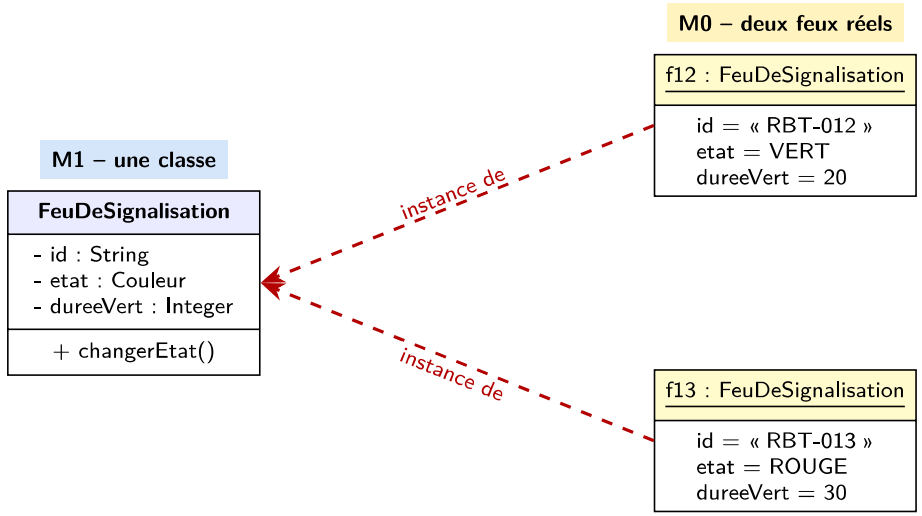
→ Il décrit tous les feux possibles, pas un seul



- Un modèle décrit **un système particulier** (logiciel, organisation, dispositif...).
- C'est une **abstraction** du réel : la classe décrit **tous** les feux.
- Chaque élément M0 est une **instance** d'un élément M1.
- Le modèle doit être **conforme** à un méta-modèle (M2).

## Analogie

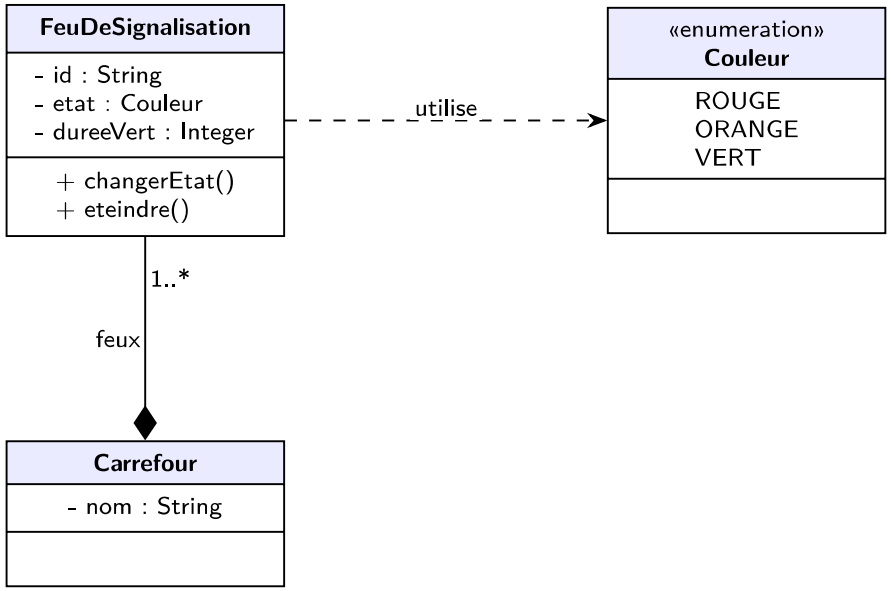
La classe est un **moule** ; les objets sont les **gâteaux** : même forme, valeurs différentes.



# Niveau M1 : le modèle

→ Vue structurelle : le diagramme de classes du feu

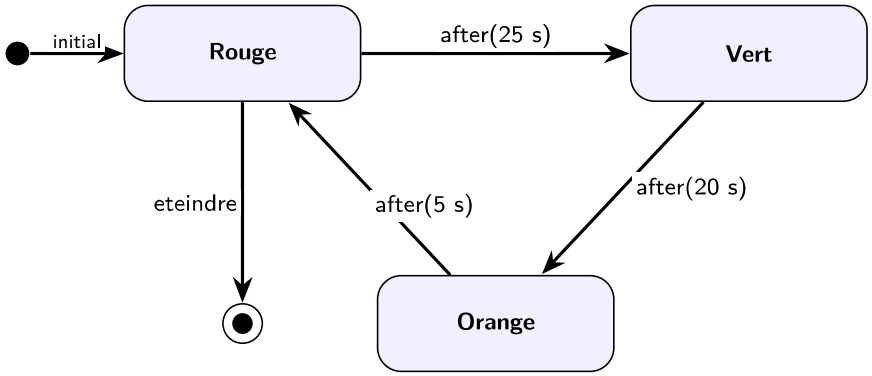





- **Classe** `FeuDeSignalisation` : attributs `id`, `etat`, `dureeVert`.
- **Opérations** : `changerEtat()`, `eteindre()`.
- **Énumération** `Couleur` = {ROUGE, ORANGE, VERT}.
- **Composition** : un `Carrefour` possède 1..* feux.

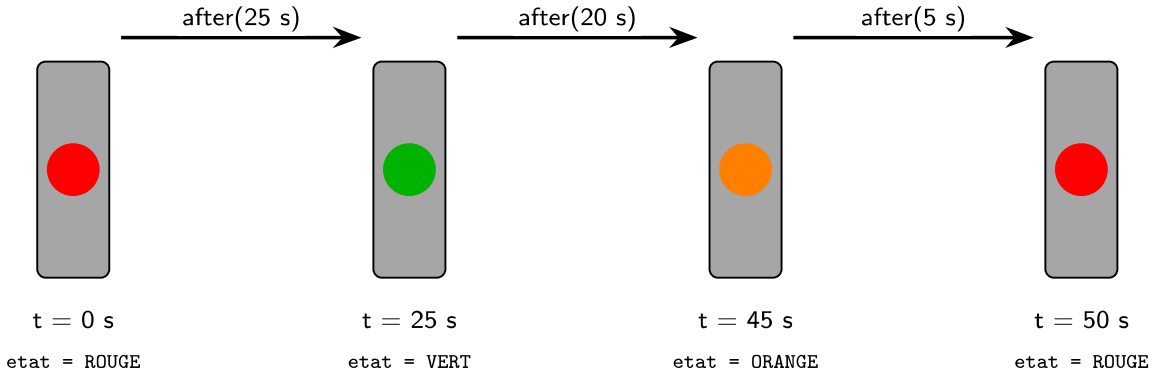
# Niveau M1 : le modèle

→ Vue comportementale : la machine à états... et son exécution (M0)




- État initial (•) : le feu démarre en **Rouge**.
- Transitions temporelles : `after(25 s)`, `after(20 s)`, `after(5 s)`.
- État final (⊙) : atteint quand on **éteint** le feu.

**Exécution de ce modèle par le feu f12 (une trace M0) :**



# Niveau M2 : le méta-modèle



Le **méta-modèle** définit les **concepts**, les **relations** et les **règles** qui permettent de construire des modèles M1. Il garantit leur **cohérence** : c'est la grammaire du langage.

> **!** Le méta-modèle UML dit qu'une classe a des attributs et des opérations, et qu'une machine à états a des états et des transitions : nos deux diagrammes sont donc **valides**.

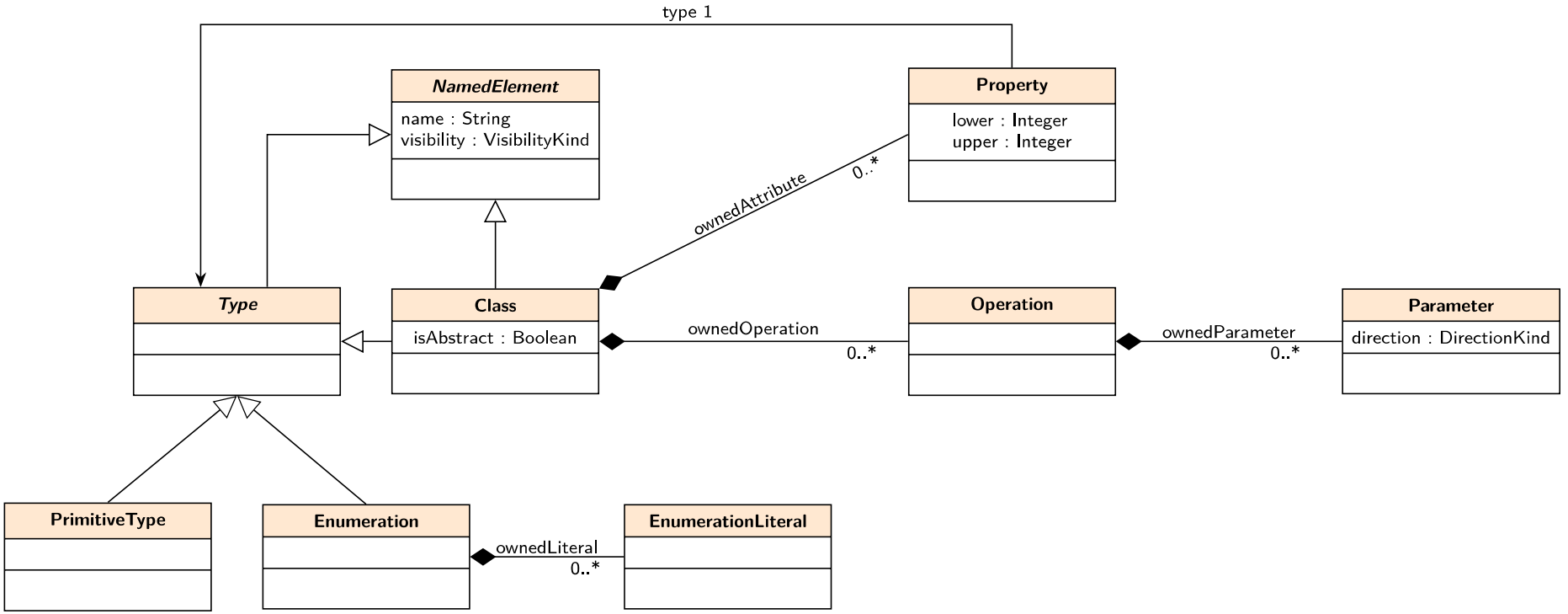
## Analogie avec la langue

- M2 = la **grammaire** : sujet, verbe, complément...
- M1 = une **phrase** correcte : « Le feu passe au vert. »
- « Vert le au passe » : **non conforme**.

# Niveau M2 : le méta-modèle



→ Méta-modèle simplifié du diagramme de classes

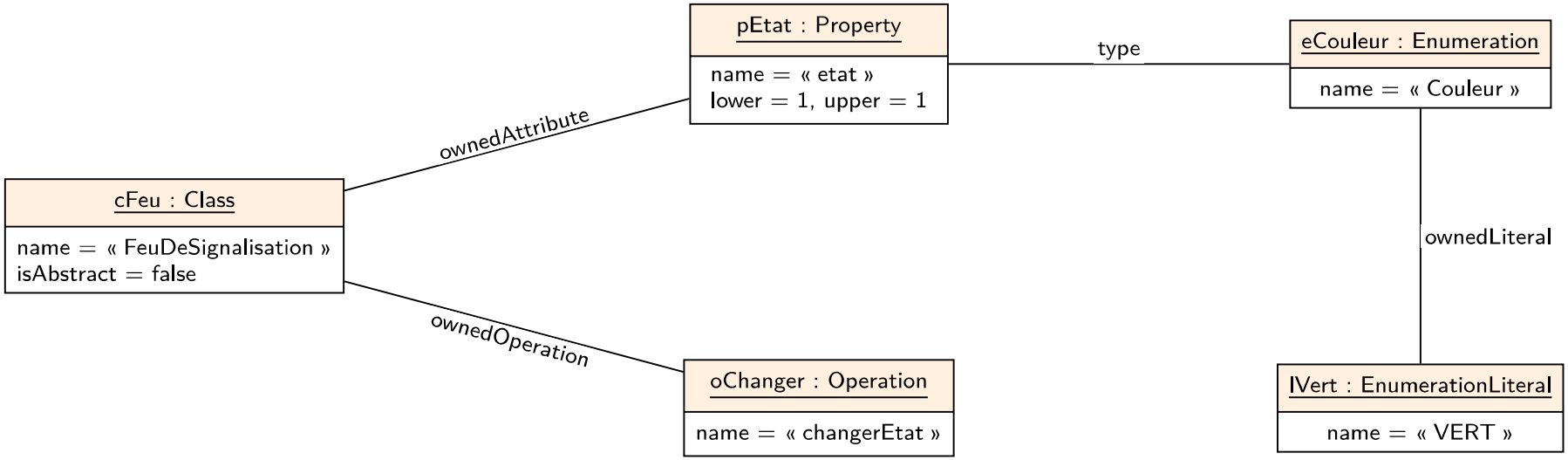



Une **Class** possède (♦) des **Property** (avec un nom, une multiplicité et exactement **1 type**) et des **Operation** (avec des **Parameter**).

# Niveau M2 : le méta-modèle

M0 | M1 | **M2** | M3

→ Le modèle M1 vu comme un ensemble d'objets du méta-modèle



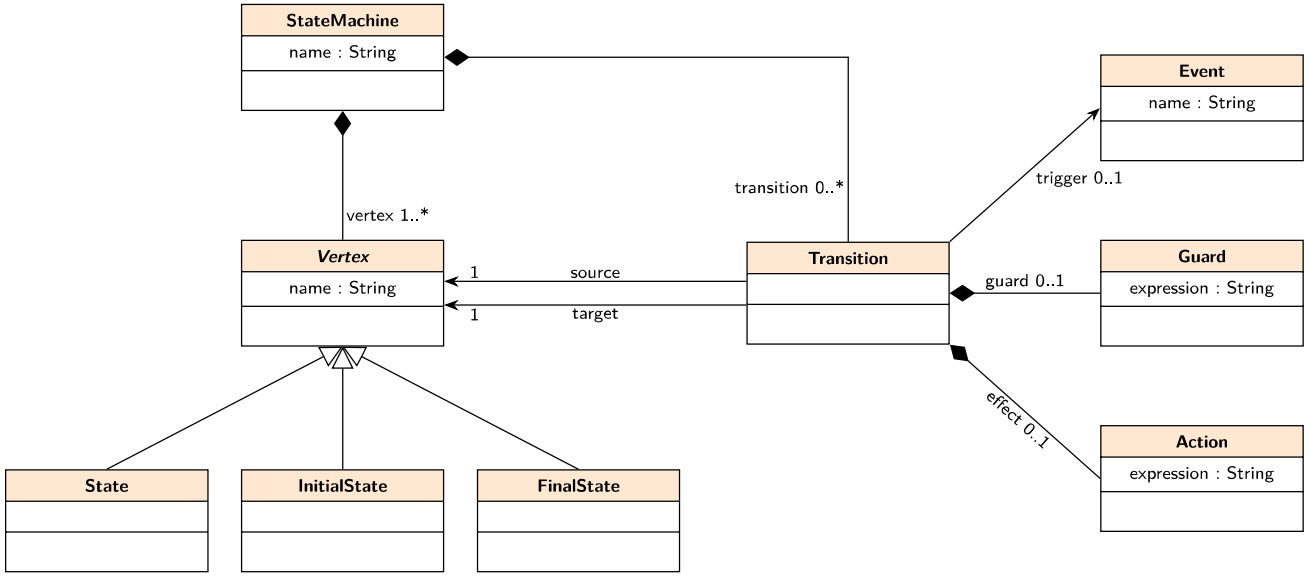

On « démonte » la classe `FeuDeSignalisation` : chaque morceau devient un **objet** dont la classe est une **méta-classe** (M2).
Le modèle M1 est à M2 ce que le monde réel M0 est à M1.

! Un diagramme est **lui-même un ensemble d'objets** : c'est ce qui permet à un outil (EMF, Papyrus) de le stocker, le parcourir et le vérifier.

# Niveau M2 : le méta-modèle



→ Méta-modèle simplifié de la machine à états




Pour le feu : `Rouge`, `Vert`, `Orange` → **State** ; le point noir → **InitialState** ; la flèche Rouge → Vert → **Transition** ; `after(25 s)` → **Event**.

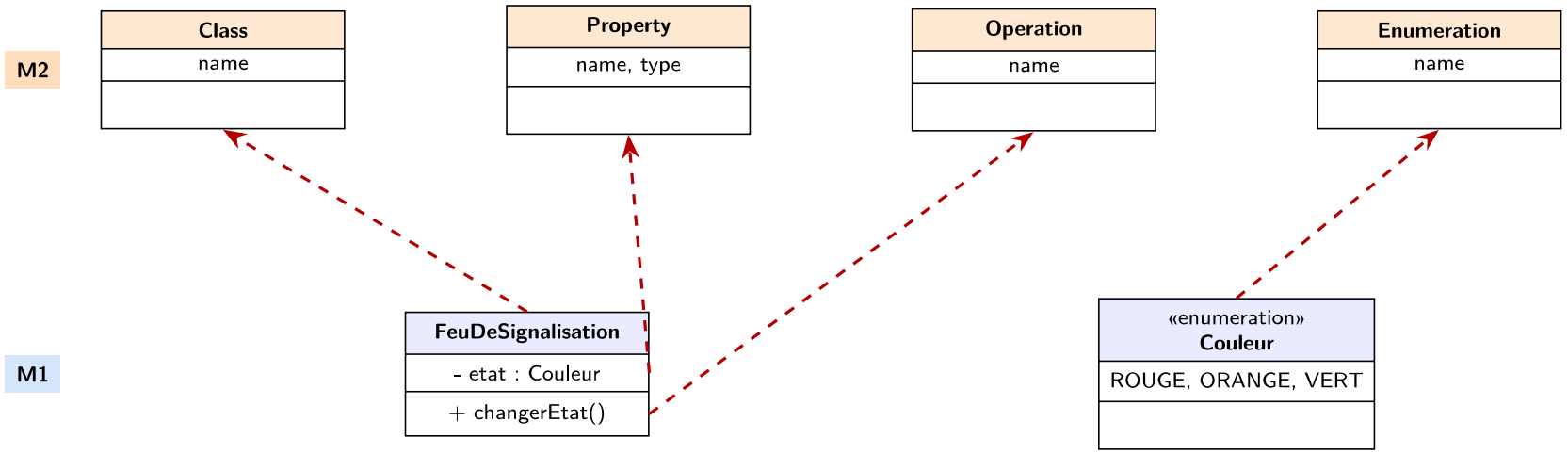
## Règles du méta-modèle (contraintes)

exactement **un** InitialState par machine ; aucune transition ne **sort** d'un FinalState ; aucune n'**entre** dans un InitialState.

# La conformité



→ Relation entre un modèle et son méta-modèle




Chaque élément du modèle est **instance** d'une méta-classe **et** toutes les contraintes du méta-modèle sont respectées ⇒ le modèle est **conforme.**

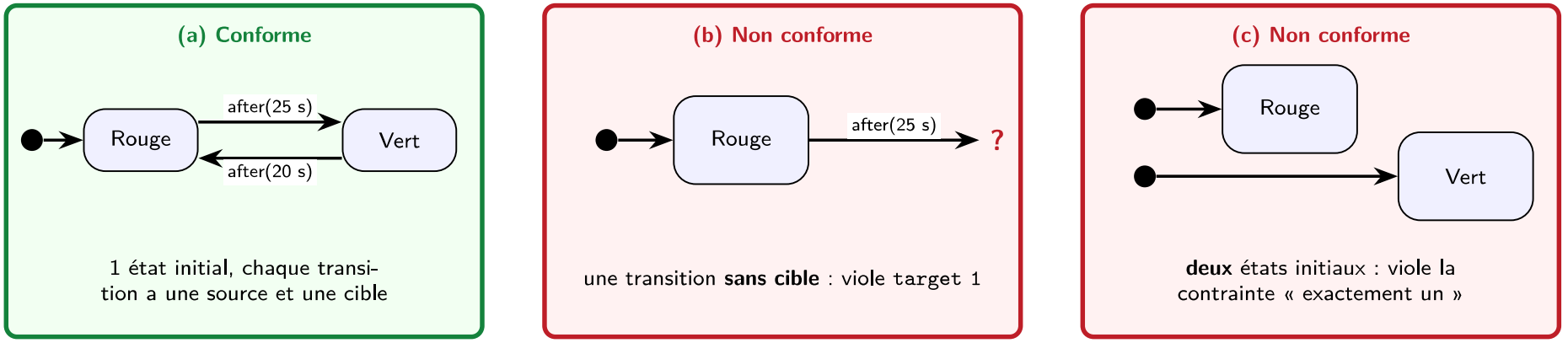
## Définition

Un modèle est **conforme** à un méta-modèle si (1) **tous** ses éléments sont instances des méta-classes **et** (2) **toutes** les contraintes du méta-modèle sont respectées.

# La conformité

→ Exemples et contre-exemples





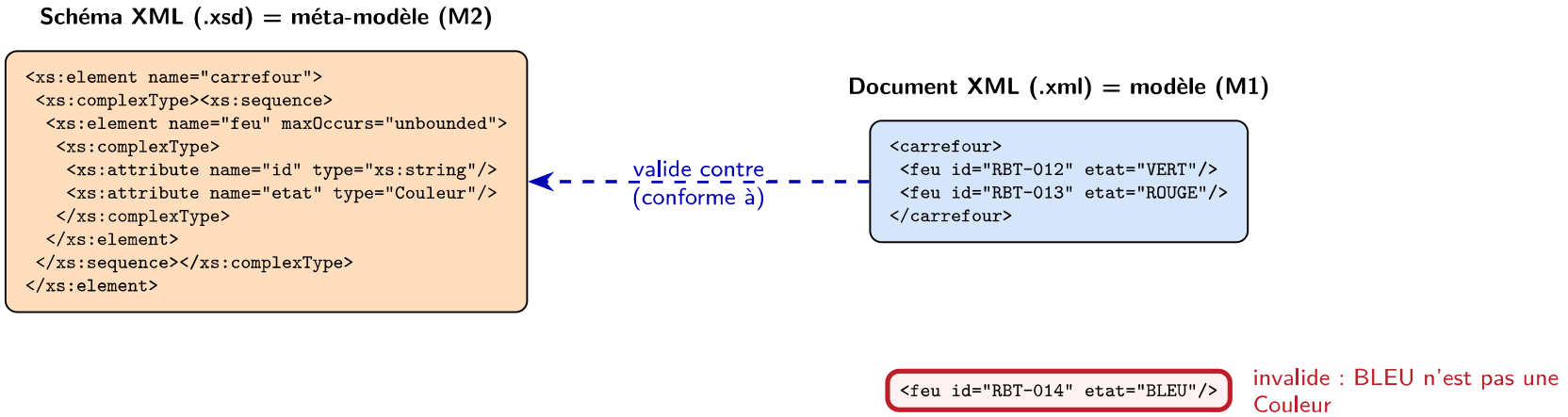

- (b) et (c) se **dessinent** très bien sur papier… mais un outil les **refuse**.
- La vérification devient **automatique** ; les règles fines s'écrivent en **OCL** (séance dédiée) :

```
context StateMachine inv UnSeulInitial:
  self.vertex->select(v | v.oclIsKindOf(InitialState))->size() = 1
```

# Un exemple hors UML

→ Schéma XML (méta-modèle) et document XML (modèle)






Le **schéma XSD** fixe les éléments et les types autorisés ; le **document XML** décrit un carrefour particulier ; le feu RBT-012 dans la rue est en **M0**.

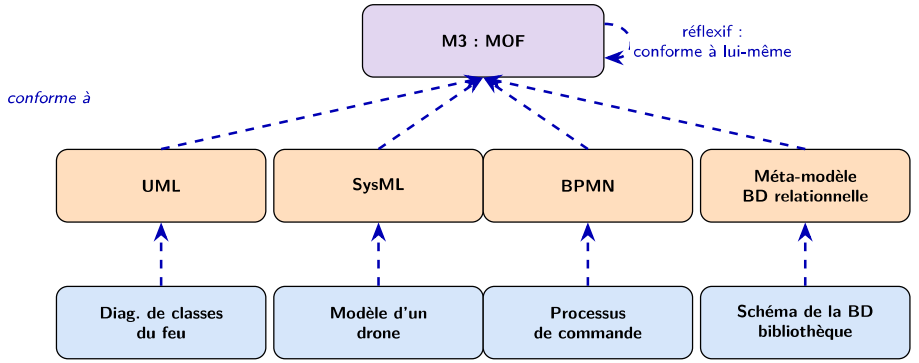
# Niveau M3 : le méta-méta-modèle

→ Le MOF (Meta Object Facility)



- Niveau **le plus abstrait** : il définit les concepts pour écrire des **méta-modèles** (UML, SysML, BPMN...).
- **Standard** de l'OMG ; concepts : Class, Property, Operation, Type, Package.
- **Réflexif** : le MOF est décrit avec le MOF, d'où l'arrêt de la pile à M3.
- Deux variantes : **EMOF** (noyau simple) et **CMOF** (complet).

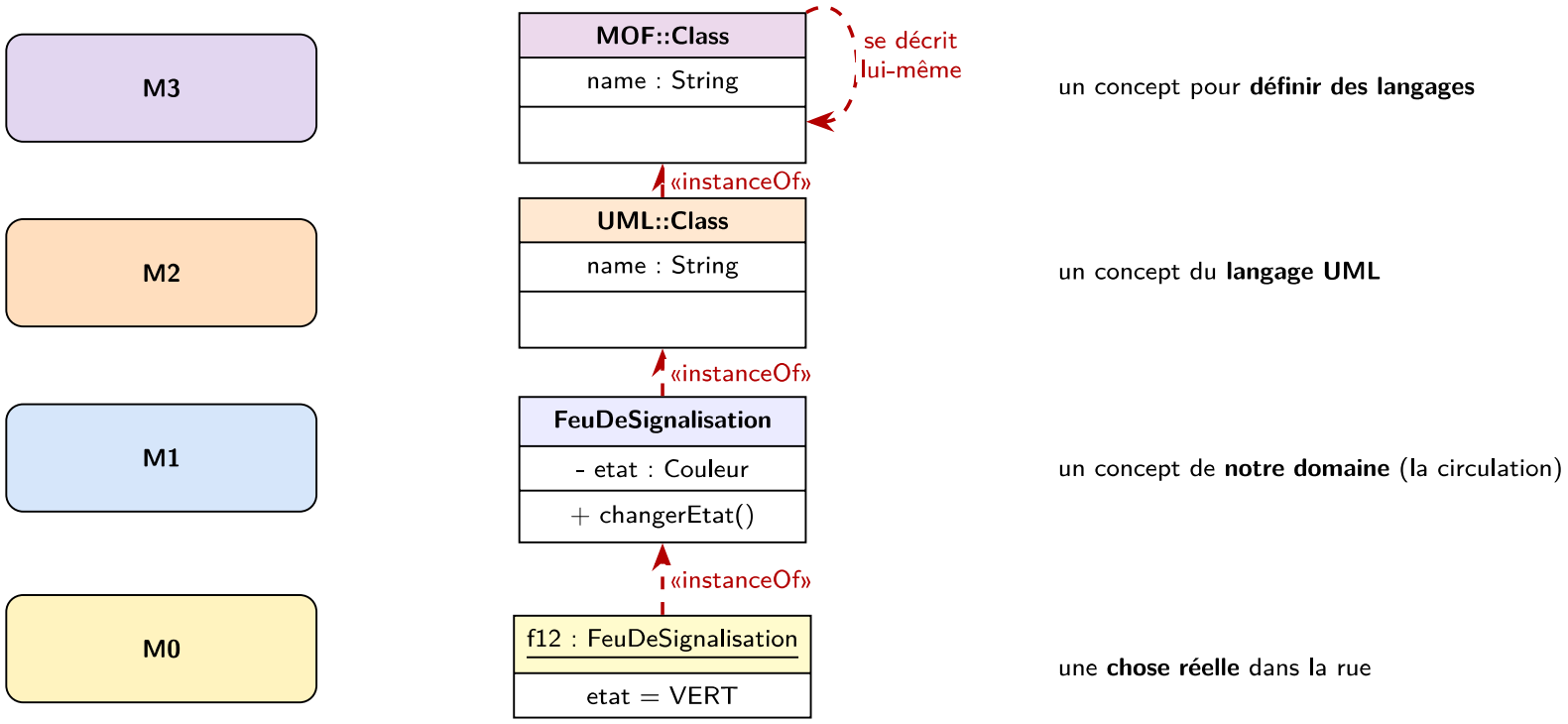
> **!** Le méta-modèle UML est un **modèle** pour le MOF, et un **méta-modèle** pour vos diagrammes.



# Niveau M3 : le méta-méta-modèle

→ La chaîne complète « instance de » du feu de signalisation



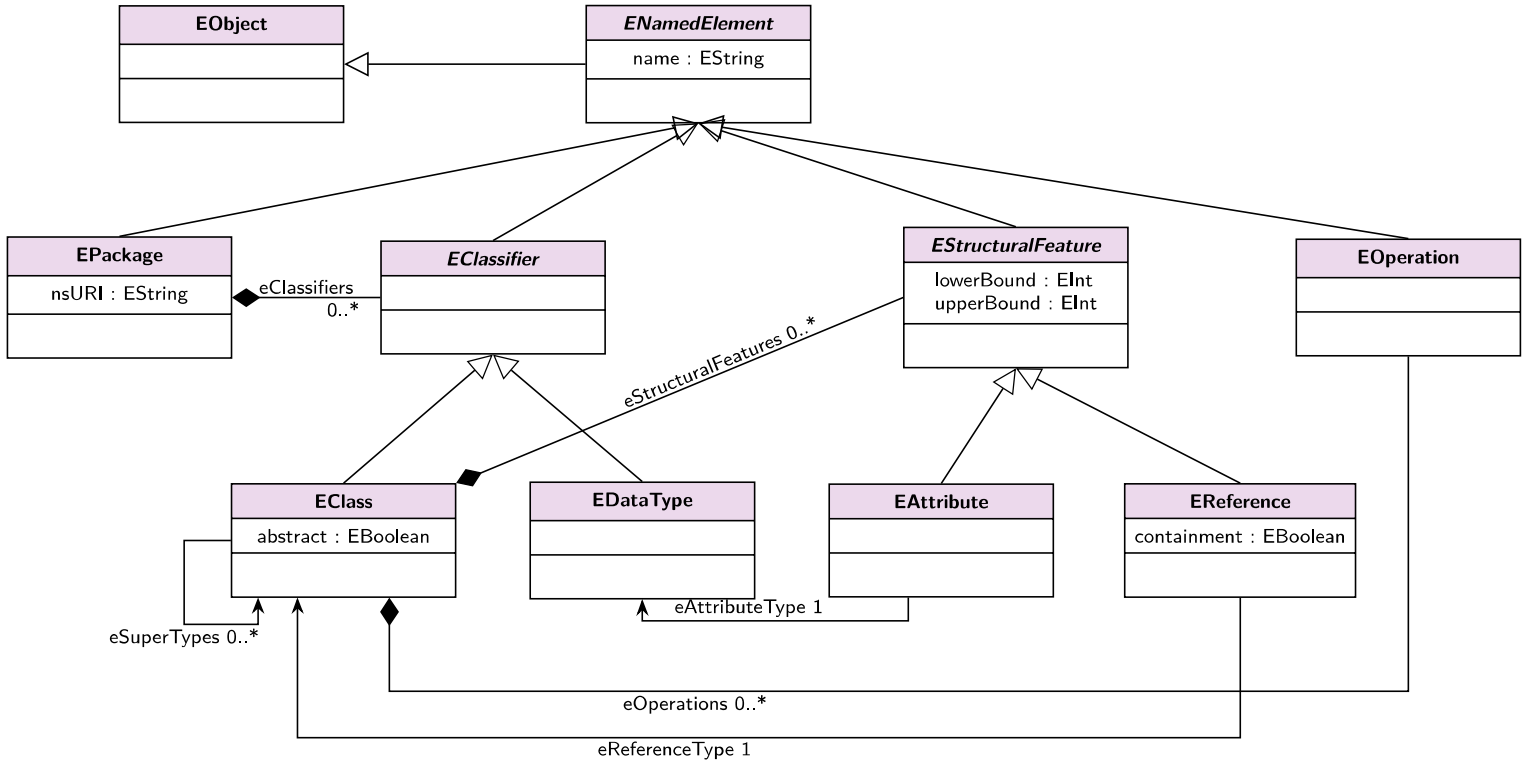



On remonte toujours la même relation : **objet** → **classe** → **méta-classe** → **méta-méta-classe**.

# Niveau M3 : le méta-méta-modèle



→ Ecore, l'implantation d'EMOF dans Eclipse (EMF)

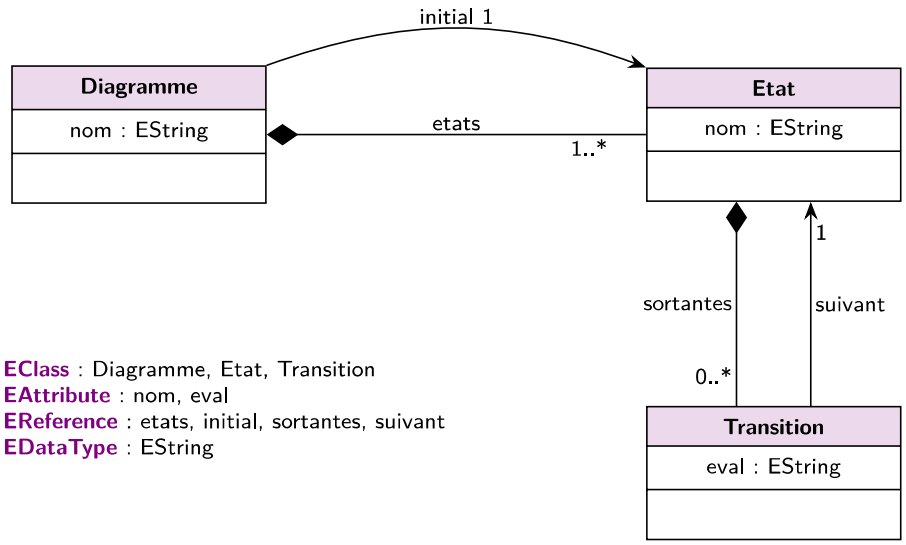



| MOF (M3) | Ecore | Exemple UML (M2) | Exemple feu (M1) |
|---|---|---|---|
| Class | EClass | Class, State | FeuDeSignalisation, Rouge |
| Property (type simple) | EAttribute | name, isAbstract | etat, dureeVert |
| Property (type classe) | EReference | ownedAttribute, source | feux |

# Niveau M3 : le méta-méta-modèle

→ Écrire un méta-modèle en Ecore, enregistrer un modèle en XMI






**EClass** : Diagramme, Etat, Transition
**EAttribute** : nom, eval
**EReference** : etats, initial, sortantes, suivant
**EDataType** : EString

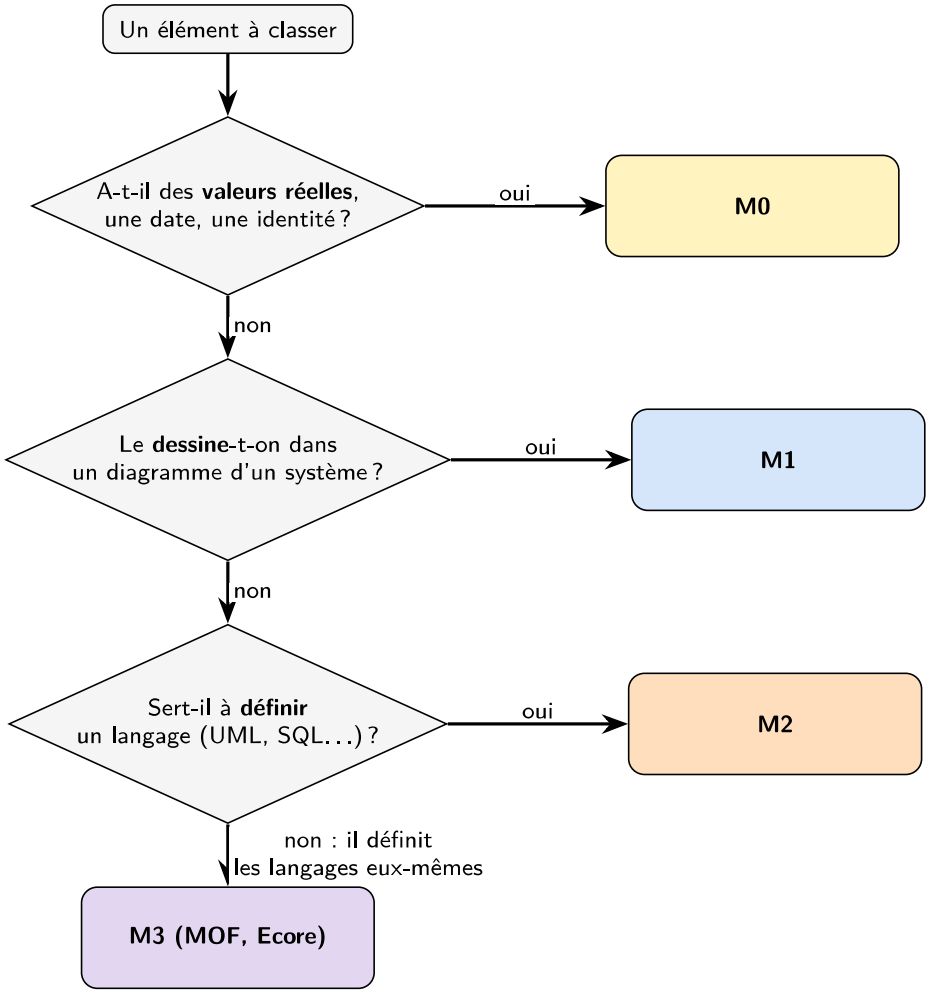
Un mini-langage de machines à états : un **Diagramme** contient des **Etat** reliés par des **Transition**.

```
<sm:Diagramme nom="Feu" initial="//@etats
    .0">
  <etats nom="Rouge">
    <sortantes eval="after(25)"
               suivant="//@etats.1"/>
  </etats>
  <etats nom="Vert"> ... </etats>
  <etats nom="Orange"> ... </etats>
</sm:Diagramme>
```

Le fichier **feu.xmi** (M1) : une balise = un objet ; les liens sont des chemins (`//@etats.1`).

# Méthode

→ Comment reconnaître le niveau d'un élément ?




**Exemples**
« la porte n° 12 fermée » → M0
la classe `Porte` → M1
la méta-classe `State` → M2
`EClass` d'Ecore → M3

# Méthode

→ Pièges fréquents

| Erreur fréquente | Ce qu'il faut dire |
|---|---|
| « La classe Livre est au niveau M0 car elle représente un vrai livre. » | La classe est en **M1** ; seul **un livre précis** est en M0. |
| « Le méta-modèle, c'est mon diagramme le plus abstrait. » | Non : il décrit **le langage** et ne contient aucun concept métier (Livre, Feu...). |
| « Une multiplicité 0..* est une méta-classe. » | Non : ce sont les **valeurs** de `lower` / `upper` d'une **Property**. |
| « M1 est instance de M0. » | C'est l'inverse : M0 est **instance de** M1, et M1 est **conforme à** M2. |
| « Il existe un niveau M4. » | Non : le MOF se décrit lui-même (réflexivité). |

**Question flash**

? Le méta-modèle UML est-il un **modèle** ?

**Question flash**

? Le méta-modèle UML est-il un **modèle** ?

**Réponse**

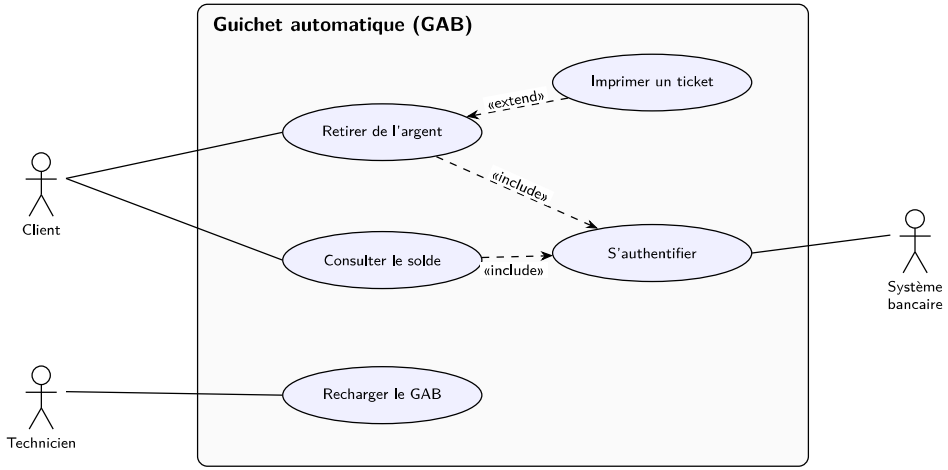
**Oui** : c'est un modèle conforme au MOF (M3). Il est « méta » seulement **par rapport à** vos diagrammes (M1).

# Exercices

*Savez-vous classer, construire et vérifier ?*

# Exercices

→ Exercice 1 : énoncé

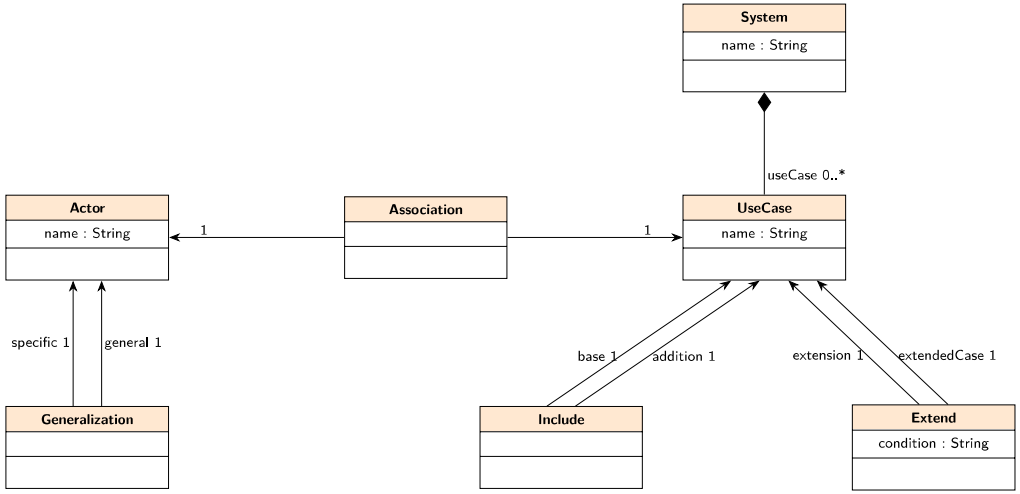



1. Que représente le schéma ci-contre ?
2. À quel niveau de l'OMG appartient-il ?
3. Modèle de conception, d'analyse, de simulation ou de communication ?
4. Quel méta-modèle permet de le **valider** ? Le dessiner (version simplifiée).
5. Donner un exemple d'élément M0 lié à ce schéma.

# Exercices

→ Exercice 1 : correction

1. Un **diagramme de cas d'utilisation** d'un GAB.
2. Niveau **M1**.
3. Modèle d'**analyse** et de **communication** : il dit **ce que** fait le système, pas **comment**.
4. Le méta-modèle des cas d'utilisation (ci-contre).
5. « Mme Amrani retire 500 DH au GAB de l'Agdal le 03/10 à 9 h 14 » : **M0**.



# Exercices



Donner un **méta-modèle** d'une base de données relationnelle :

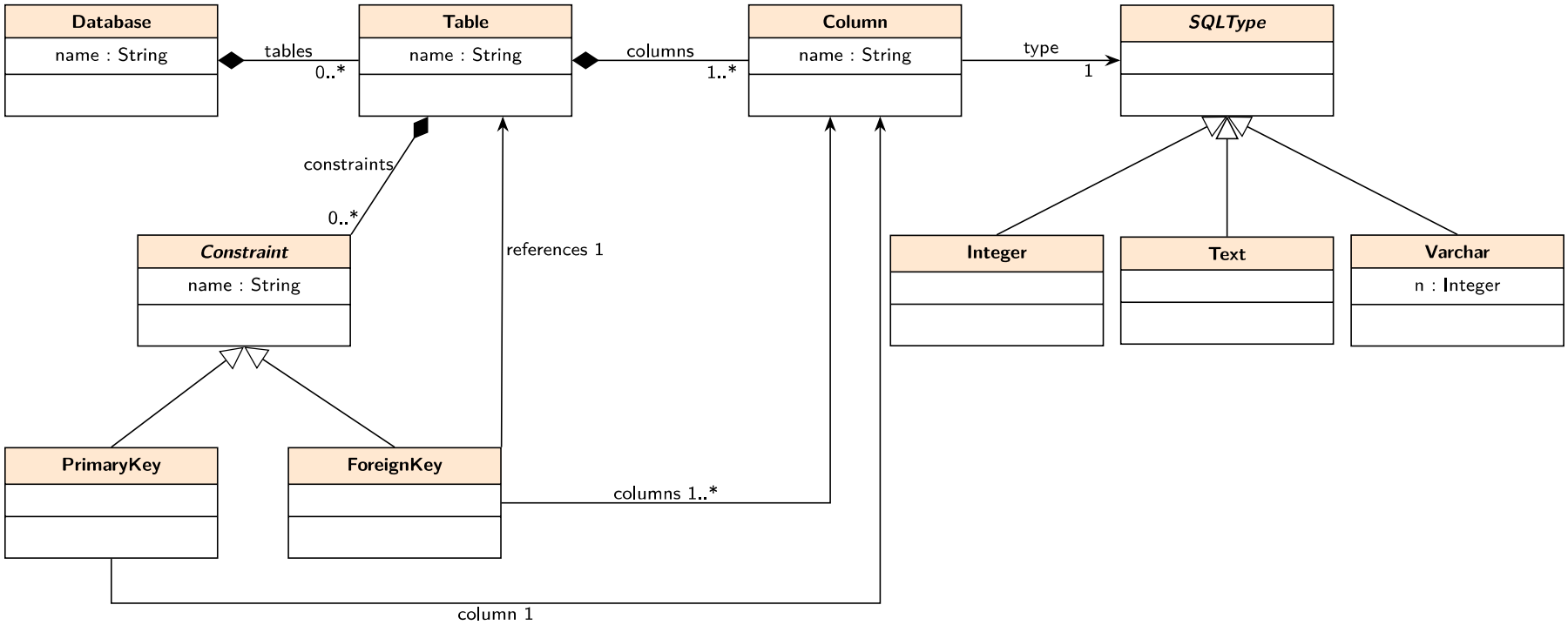
- Une base contient des **tables** ; une table comporte des **colonnes** ; tous ont un **nom**.
- Une colonne a un **type SQL** : `Integer`, `Text` ou `Varchar(n)`.
- Une table a des **contraintes** nommées : **Primary Key** (une seule colonne) ou **Foreign Key** (une ou plusieurs colonnes, référence une autre table).

## Méthode conseillée

1. **Noms** de l'énoncé → méta-classes.
2. « contient / comporte » → **compositions**.
3. « de deux sortes » → **héritage** (méta-classe abstraite).
4. « se caractérise par » → **attributs**.

# Exercices

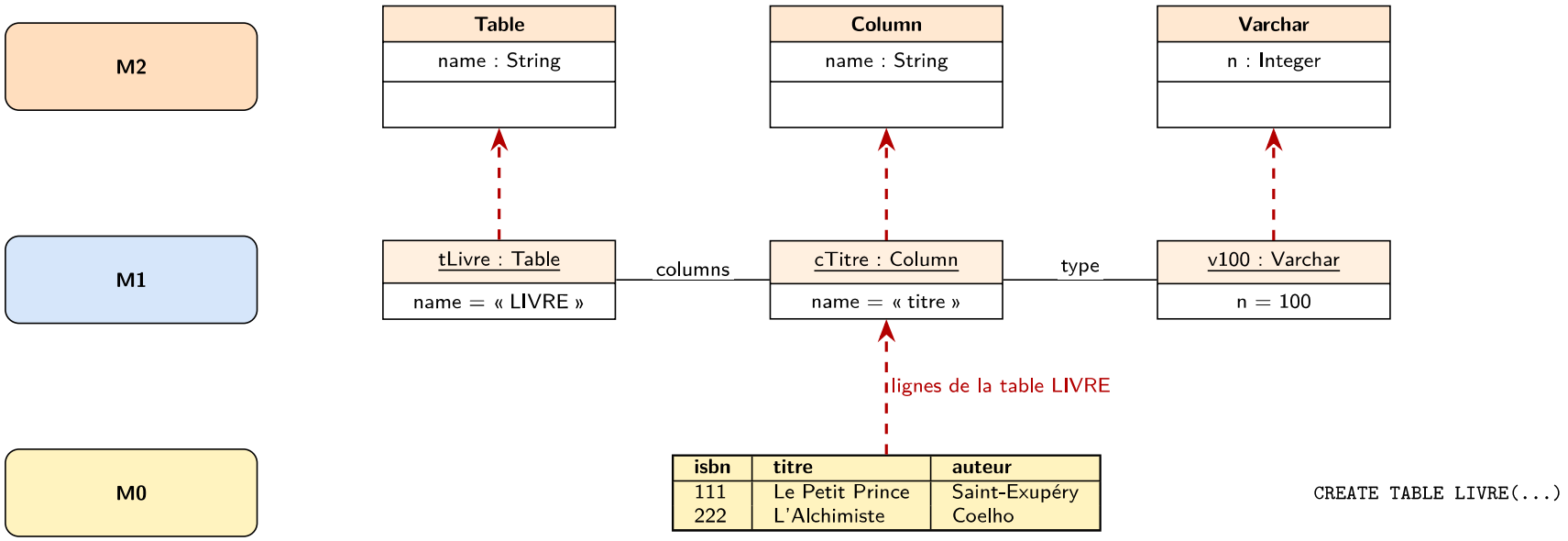
→ Exercice 2 : correction




`SQLType` et `Constraint` sont **abstraites** : on n'instancie que `Integer`, `Text`, `Varchar`, `PrimaryKey`, `ForeignKey`.

# Exercices

→ Exercice 2 : la base de la bibliothèque sur trois niveaux




La table **LIVRE** est un modèle (M1) conforme au méta-modèle relationnel (M2) ; ses **lignes** sont des données réelles (M0).

**Une bibliothèque gère :**

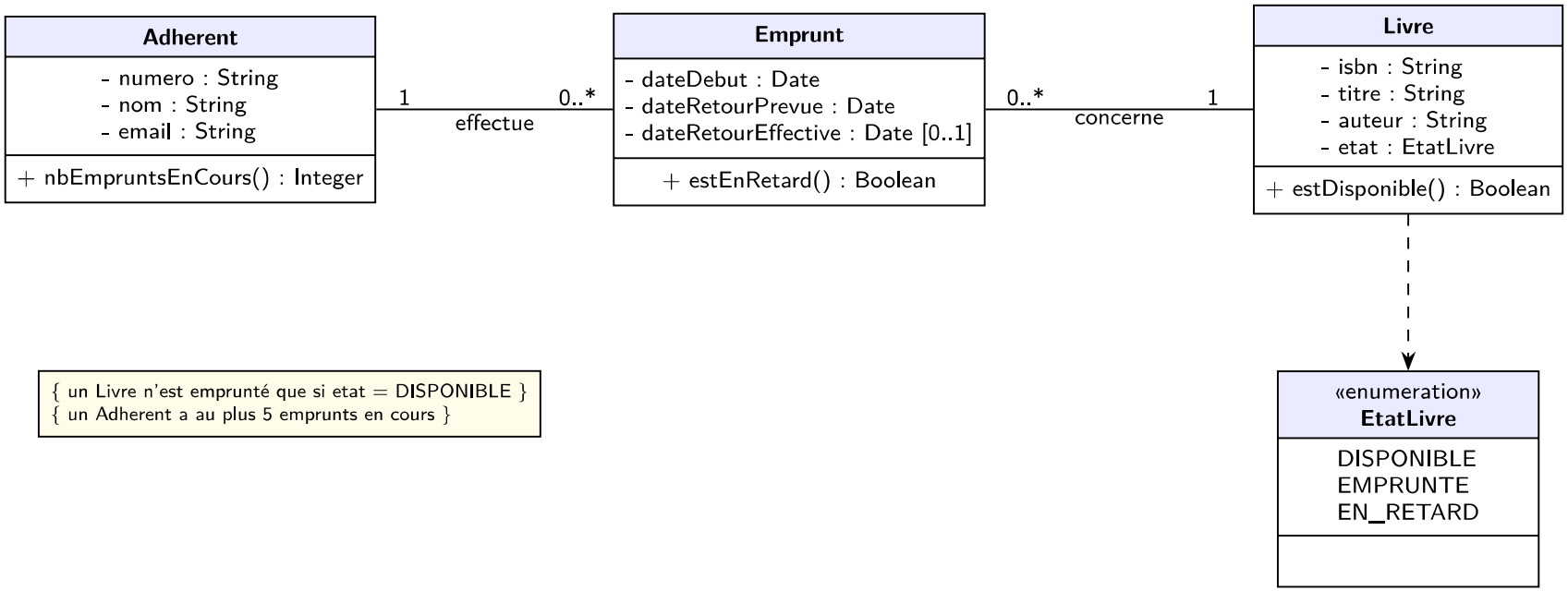
- des **livres** : ISBN, titre, auteur, état (disponible, emprunté, en retard) ;
- des **adhérents** : numéro, nom, e-mail ;
- des **emprunts** : date de début, date de retour prévue, date de retour effective (optionnelle) ;
- un livre n'est emprunté que s'il est **disponible** ; au plus **5** emprunts par adhérent.

**Travail à faire**

1. Dessiner le diagramme de classes.
2. Donner la **méta-classe** UML de chaque élément.
3. Niveau OMG de :
   - votre diagramme ;
   - le concept « Class » d'UML ;
   - un livre physique ;
   - le MOF.

# Exercices

→ Exercice 3 : correction




- Adherent, Emprunt, Livre → **Class** ; isbn, titre. . . → **Property** ; EtatLivre → **Enumeration** ; effectue, concerne → **Association** ; estDisponible() → **Operation**.
- Diagramme → **M1** ; « Class » → **M2** ; livre physique → **M0** ; MOF → **M3**.

**Question flash**

?

Classer : (a) le langage SQL ; (b) la table LIVRE ; (c) la ligne (111, « Le Petit Prince ») ; (d) l'état `Orange` ; (e) la méta-classe `Transition` ; (f) `EReference` ; (g) le feu n° 13 ; (h) le fichier `carrefour.xsd`.

**Question flash**

?

Classer : (a) le langage SQL ; (b) la table LIVRE ; (c) la ligne (111, « Le Petit Prince ») ; (d) l'état `Orange` ; (e) la méta-classe `Transition` ; (f) `EReference` ; (g) le feu n° 13 ; (h) le fichier `carrefour.xsd`.

**Réponse**

(a) **M2** · (b) **M1** · (c) **M0** · (d) **M1** · (e) **M2** · (f) **M3** · (g) **M0** · (h) **M2** (un schéma XSD joue le rôle de méta-modèle).

# À retenir

## À retenir

**Modèle**

Représente un système, le simplifie et sert un objectif.

**IDM**

Le modèle devient productif : on le vérifie, on le transforme, on génère du code.

**M0**

Le réel : objets concrets, datés, avec des valeurs.

**M1**

Le modèle d'un système ; M0 en est une instance.

**M2**

Le méta-modèle (le langage) ; M1 lui est conforme.

**M3**

MOF / Ecore : définit les langages et se décrit lui-même.

### Prochaines séances

TD : les niveaux M0 $\rightarrow$ M3 sur des études de cas. TP : construire un méta-modèle avec EMF / Ecore, l'instancier, le vérifier avec OCL et l'exécuter.

# Glossaire

| Terme | Définition courte |
|---|---|
| Modèle / Méta-modèle | abstraction d'un système / modèle qui définit un langage de modélisation |
| Méta-classe | concept d'un méta-modèle (Class, State, Table...) |
| Instance de | relation entre un élément et le concept qui le définit |
| Conformité | un modèle respecte toutes les règles de son méta-modèle |
| IDM / MDE | Ingénierie Dirigée par les Modèles / Model-Driven Engineering |
| MOF / Ecore | langages de méta-modélisation (M3) de l'OMG / d'Eclipse |
| UML, DSL | langage de modélisation généraliste ; langage dédié à un domaine |
| XMI, OCL | format XML d'échange des modèles ; langage de contraintes |